Cibersegurança - Segurança em Software (Módulo 2)
Trabalho Prático - Parte 2

Objetivos:
- Usar a ferramenta CodeQL para realizar análise estática de código (SAST), em ambiente local e em repositórios GitHub

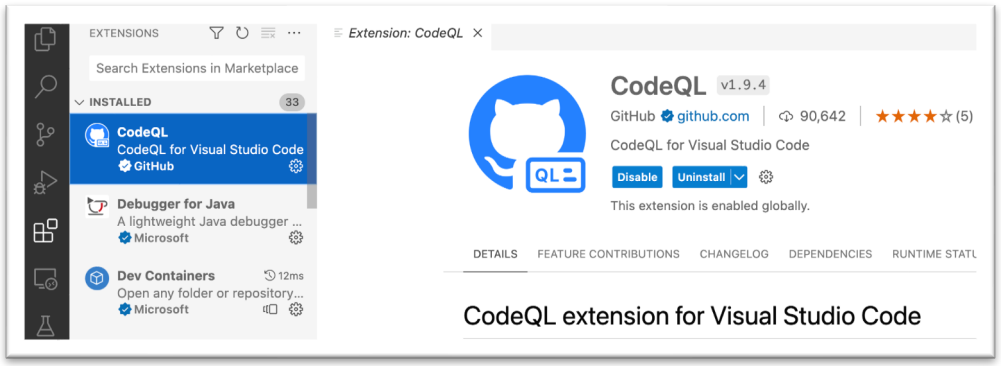
Entrega:
- Respostas às questões do enunciado
- Evidências de realização das tarefas A.5.c, A.5.f, A.5.g, C.1, C.3

A. CodeQL – Execução local

1. Instale localmente as ferramentas da plataforma CodeQL através da aplicação Visual Code:

   a. Instale a aplicação Visual Code: https://code.visualstudio.com

   b. Instale a versão mais recente da extensão CodeQL do Visual Code.



   Passará a ter este icon:



   c. Instale as ferramentas da linha de comandos do sistema CodeQL, escolhendo a versão correta para o seu sistema operativo: https://github.com/github/codeql-cli-binaries/releases.

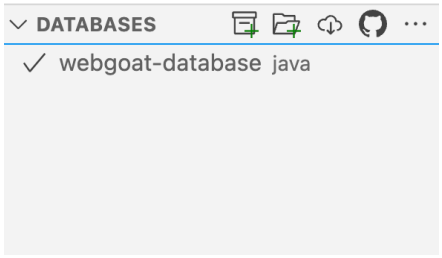
   d. Faça clone do *starter kit* para uma diretoria no seu computador: https://github.com/github/vscode-codeql-starter.

   e. Faça clone do repositório da aplicação de aprendizagem WebGoat: https://github.com/WebGoat/WebGoat. A aplicação WebGoat é uma aplicação web deliberadamente insegura, para uso em contexto de treino sobre segurança no software.

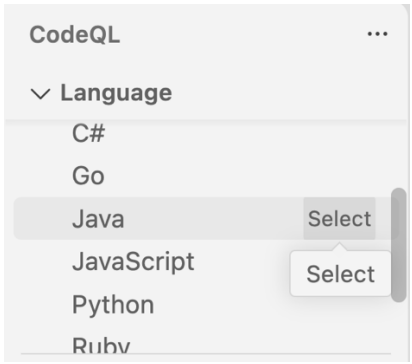
2. Crie a base de dados CodeQL a partir do código fonte da WebGoat (comando executado na diretoria de base do código fonte). Tenha em conta que este comando irá tentar compilar a aplicação WebGoat. Para tal, precisa de ter instalado no computador uma JVM compatível com a versão Java (<java.version>) indicada no ficheiro pom.xml.

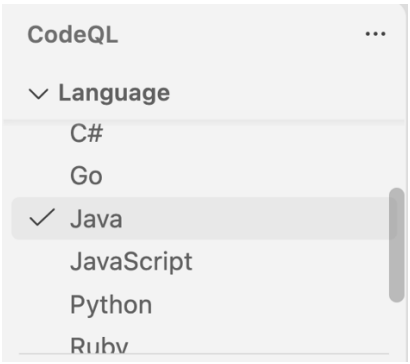
```
codeql database create webgoat-database --language=java --overwrite
```

3. Abra o *workspace* do *starter kit* no Visual Code e carregue a base de dados para o CodeQL no Visual Code usando a opção "Choose database from folder". Confirme que a base de dados foi carregada corretamente:



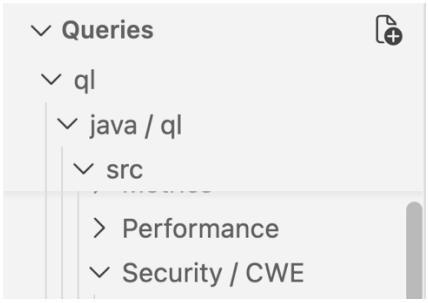
4. Selecione a linguagem Java para filtrar por interrogações que analisem código fonte nesta linguagem:





5. Pretende-se avaliar vulnerabilidades do tipo *Use of Hard-coded Credentials* – CWE 798 (https://owasp.org/www-community/vulnerabilities/Use_of_hard-coded_password, https://cwe.mitre.org/data/definitions/798.html):

    a. Encontre a interrogação QL correspondente no *workspace* do *starter kit* (`HardcodedPasswordField.ql`)



b. Analise a interrogação:

   i. Leia o cabeçalho da interrogação: que valor tem a propriedade `@precision` (mais informações em https://codeql.github.com/docs/writing-codeql-queries/metadata-for-codeql-queries/)?

   ii. Com base no código da interrogação, explique sucintamente como decide se um campo contém uma *password*.

   iii. Trata-se de análise de fluxo de dados?

c. Excute a interrogação (procure o icon ao lado do ficheiro com extensão .ql ). Na lista de resultados, escolha a entrada referente ao ficheiro `JWTRefreshEndpoint.java`. Para que serve, no código, a *password* assinalada?

d. No mesmo ficheiro existe outro segredo escrito no código-fonte que a interrogação **não** assinala. Encontre-o e:

   i. Descreva em que consiste a estrutura JWT (`https://jwt.io`) e qual a operação criptográfica que depende desse segredo;

   ii. Explique porque é que a interrogação não o detetou. *Sugestão:* a partir da interrogação, siga a definição da classe `PasswordVariable` (no Visual Studio Code: *Go to Definition*);

   iii. Classifique esta situação como falso positivo ou falso negativo.

e. No caso de o JWT ser usado para verificar a identidade e as características do utilizador, que tipo de ações pode um atacante realizar se conhecer o segredo encontrado na alínea d.?

f. Pretende-se analisar vulnerabilidades do tipo path transversal - CWE 22 (https://owasp.org/www-community/attacks/Path_Traversal, https://cwe.mitre.org/data/definitions/22.html):

   Encontre a interrogação QL correspondente no workspace do starter kit e analise o código fonte representado na base de dados configurada no ponto anterior.

   Identifique o resultado referente ao ficheiro `ProfileUploadBase.java` e descreva o vetor de ataque (isto é, de que forma a vulnerabilidade pode ser explorada).

g. Considere a seguinte alteração ao código fonte da aplicação WebGoat `org.owasp.webgoat.lessons.pathtraversal.ProfileUploadBase`, método `execute()`. Volte a gerar a base de dados e verique se o CodeQL deixa de assinalar a vulnerabilidade no método `execute()`:

```
// Verificar caminho canónico
var uploadedFile = new File(uploadDirectory, fullName);
if (!uploadedFile
      .getCanonicalPath()
      .startsWith(uploadDirectory.getCanonicalPath() + File.separator)
   )
{
   return failed(this).feedback("path-traversal-rejected").build();
}
```

h. Indique a categoria da OWASP Top 10:2025 onde esta vulnerabilidade se insere e o requisito da OWASP ASVS 5.0 que este código viola, no âmbito do controlo v5.0.0-5.3.

B. Aplicação Web JuiceShop

Nas secções seguintes, a análise estática é aplicada também à aplicação web Juice Shop. Esta aplicação tem erros de programação propositados, que podem ser usados para obter conhecimento sobre vulnerabilidades em geral e em aplicações web em particular. A arquitetura Juice Shop inclui um *frontend* desenvolvido em Angular, um *backend* desenvolvido em node.JS e o uso de bases de dados relacionais e não relacionais, como ilustrado na Figura 1.

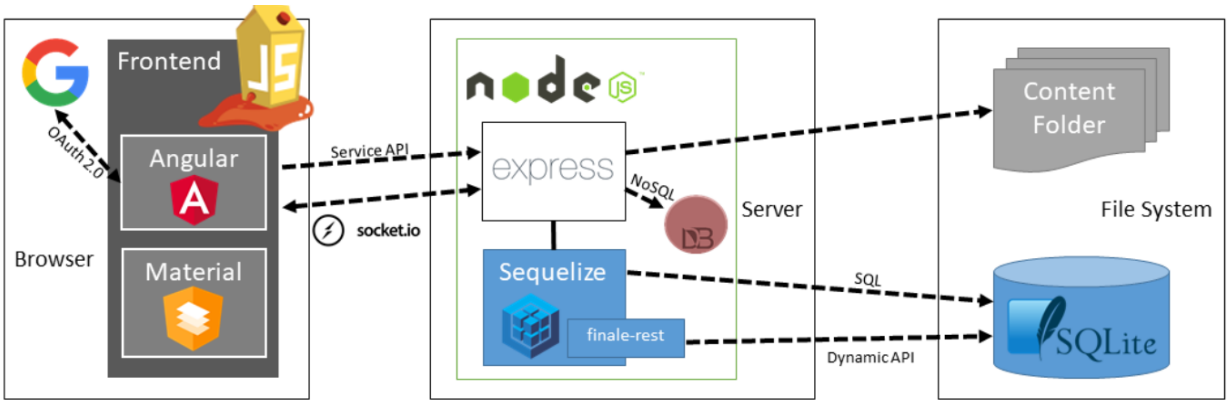


Figura 1: Arquitetura da Aplicação Juice Shop

Crie um fork da aplicação Juice Shop (https://github.com/juice-shop/juice-shop) para um repositório público do seu grupo (indique o URL do repositório no documento de entrega).

C. CodeQL – execução no Github

1. Num *fork*, as GitHub Actions vêm desativadas. No separador **Actions** do seu *fork*, confirme que pretende ativar os *workflows*.

   O repositório já inclui o *workflow* de análise CodeQL: `.github/workflows/codeql-analysis.yml` (nome "CodeQL Scan"), que corre a cada evento `push` e `pull_request`. Analise o *workflow* e altere o gatilho para que a análise seja acionada manualmente (`workflow_dispatch`): https://docs.github.com/en/actions/using-workflows/events-that-trigger-workflows#workflow_dispatch

   Notas:
   - O repositório inclui outros *workflows* (testes, *lint*, publicação). Podem correr ou falhar no seu *fork*; ignore-os, porque não fazem parte deste trabalho.
   - **Não** ative o *Default setup* do *code scanning* (Settings → Code security). Entra em conflito com o *workflow* que já existe no repositório.

2. Execute a ação manualmente. Nos *logs* de saída, identifique a etapa e o comando usados pela ação CodeQL para inicializar a base de dados com metadados sobre o código-fonte. Onde fica a base de dados que resulta da análise?

3. Considere a vulnerabilidade CWE-89 "Improper Neutralization of Special Elements used in an SQL Command" (https://cwe.mitre.org/data/definitions/89.html). Procure a entrada "*Database query built from user-controlled sources*" no arquivo `routes/search.ts` na lista de vulnerabilidades detetadas pelo Github Action CodeQL. Justifique o motivo do CodeQL identificar esse código como vulnerável. Inclua *informações de source* e *sink*. (mais informações sobre SQL Injection podem ser encontradas aqui https://owasp.org/www-community/attacks/SQL_Injection).